

Regex examples

1. All strings consisting of 0's and 1's (binary digits) with an even number of 0s
 $1^*(01^*01^*)^*$
2. Set of strings where each pair of zeroes is followed by a pair of ones.
 $(1+01+0011)^*(0+\varepsilon)$
3. The set of strings not containing 101.
 $0^*(1+00^+)^*0^*$
4. The set of binary numbers divisible by 4.
 $(0+1)^*00+0$
5. Let $\Sigma = \{a, b\}$ and let $L = \{w \in \Sigma^* \mid w \neq \varepsilon \text{ and the first and last character of } w \text{ are the same}\}$. Write a regular expression for L. (This question can also be thought as the number of a's and b's differ by at most one)
 $a \cup b \cup a \Sigma^* a \cup b \Sigma^* b$

This says "either match the single characters a and b, or match something that starts and ends with a with any number of characters in the middle, or match something that starts and ends with b with any number of characters in the middle."

You can condense this a bit using the ? operator:

$a(\Sigma^*a)? \cup b(\Sigma^*b)?$

This means "if you see an a, either you're done, or you're going to see some number of characters and then another a" (and the same for b).

6. Let $\Sigma = \{a, b\}$ and let $L = \{w \in \Sigma^* \mid w \text{ is a nonempty string whose characters alternate between a's and b's}\}$. Write a regular expression for L.

$a(ba)^*b? \cup b(ab)^*a?$

Regex examples

If the characters alternate, they either start with an a or they start with a b. If they begin with a, then we'll then have some number of copies of the string ba, optionally ending with a final b. If they begin with b, then we'll then have some number of copies of the string ab, optionally ending with a final a.

7. All strings in which every run of 0s has length at least 3.

$(1 + 0000^*)^*$

8. All strings in which every substring 000 appears after every 1.

$(1+01+001)^*0^*$

9. All strings in which 1 does not appear after a substring 000.

(Intuition) We can have a maximum of two 0s before any 1. So we can split the problem into two substrings x and y and write it as xy. x represents all substrings with 1s where there are less than three 0s before a 1. y represents substrings containing only 0s.

(Strategy) For x, we can have either no 0 before a 1 or one 0 before a 1 or two 0s before a 1. This can be represented by $(1 + 01 + 001)^*$. For y, we can use 0 since it represents zero or more 0s.

(Result) $(1 + 01 + 001)^*0^*$

10. All strings containing at least two 0s and at least one 1.

There are three possibilities for how such a string can begin:

- Start with 00, then any number of 0s, then 1, then anything.
- Start with 01, then any number of 1s, then 0, then anything.
- Start with 1, then a substring with exactly two 0s, then anything.

All together:

$000^*1(0+1)^* + 011^*0(0+1)^* + 11^*01^*0(0+1)^*$